50 — Defensive Disclosure Strategy & Publication Procedure

v1.2 — making the lineage function as prior art

Function: Turn the v1.0 document lineage into a legally useful, dated, examiner-discoverable public record, and set up the repository and archival pipeline so every future version inherits the same properties automatically.

Standing caveat: this is engineering-process guidance, not legal advice. The strategy below reflects widely accepted prior-art practice; a one-hour consultation with an IP attorney to review the final abstract and disclosure statement is cheap insurance if the stakes ever rise.

---

1. What a defensive publication must actually achieve

A disclosure defeats a later patent claim only if it satisfies three properties, and it defeats it *in practice* only if it satisfies a fourth:

Property	What it means	How this project satisfies it
Public accessibility	Any interested member of the public could find and read it without restriction	Public GitHub repo + Zenodo open-access record; no login walls, no embargo
Verifiable date	A third party attests to *when* it became public — self-asserted dates (git commit timestamps, file metadata) are forgeable and carry little weight alone	Zenodo/DataCite DOI registration date; Software Heritage and Internet Archive snapshots; optional cryptographic timestamp
Enablement	A person of ordinary skill could practice the disclosed subject matter without undue experimentation	Already strong: synthesis routes with reagent masses, materials tables, sizing derivations, state chains, test procedures. Keep this density in every release
Examiner discoverability	Patent examiners search classification codes, keyword databases, and Google Patents/Scholar — not GitHub. Prior art that exists but is never found doesn't block anything at the examination stage (only later, expensively, in litigation)	Metadata discipline (§5), standard-terminology abstract, optional TDCommons deposit (§7)

One consequence worth stating plainly: publication permanently bars *your own* future patenting of what's disclosed, in essentially every jurisdiction (the US 12-month grace period is the partial exception). That is the stated intent of this project — this document just makes it official that the bar is deliberate.

2. The venue stack (layered, like everything else here)

No single venue does all four jobs. The stack:

Layer	Venue	Role	Cost
Living record	GitHub public repository	Discoverability, issues, forks, ongoing lineage; the canonical working copy	free
Immutable dated record	**Zenodo** (CERN-operated, DataCite DOI)	The prior-art anchor: files immutable after publication, third-party datestamp, DOI citable forever, EU-institution-backed longevity	free
Independent mirror #1	Software Heritage	Automatic long-term archive of the git history itself (commits, not just tarballs)	free
Independent mirror #2	Internet Archive (Wayback)	Snapshot of the rendered GitHub pages and the Zenodo record page	free
Examiner channel (optional)	TDCommons (free) and/or Research Disclosure journal (paid, ~$120+/item)	Databases examiners actually search; Research Disclosure is part of the PCT minimum documentation set	$0 / ~$120–200
Cryptographic belt-and-braces (optional)	OpenTimestamps on the release tarball hash	Bitcoin-anchored proof-of-existence; costs nothing, settles any future dispute about tampering	free

The first four take an afternoon. The examiner channel is the only judgment call: for a niche mechanical/HVAC domain, a well-metadata'd Zenodo record plus Google Scholar indexing catches most searches; TDCommons is free and adds direct examiner reach, so it is recommended; Research Disclosure is the gold standard and worth the fee only if you learn of active patenting in this space.

3. Repository formation procedure

3.1 Structure

```
slice-cooling/          # pick a descriptive, searchable name
├── README.md                    # existing, + DOI badge + disclosure statement (§3.4)
├── docs/
│   ├── 00_platform_basis.md … 40_findings_register.md
│   ├── 50_defensive_disclosure_plan.md
│   └── parameter_register.xlsx      # the quantitative register (§3.5)
├── diagrams/                    # the SVG set
├── rendered/                    # PDF renders of every doc (§3.3)
│   └── executive_summary.pdf
├── scripts/                     # render pipeline + register generator (MIT)
├── LICENSES/
│   ├── CERN-OHL-P-2.0.txt
│   ├── CC-BY-4.0.txt
│   └── MIT.txt
├── LICENSE.md                   # scope map: which license covers what (§3.2)
├── CITATION.cff                 # citation box on GitHub
└── .zenodo.json                 # controls Zenodo metadata (§4.3)
```

3.2 The tri-license scope map

Zenodo's GitHub integration handles multiple license files badly — if it finds more than one, it uses whichever sorts first alphabetically, and a malformed license text fails the whole deposit. Two rules:

1. Full verbatim license texts live in `LICENSES/`, **not** in a root `LICENSE` file. The root gets `LICENSE.md`, a short human map: *hardware designs and specifications → CERN-OHL-P v2; documentation and diagrams → CC-BY-4.0; scripts and code → MIT*.
2. The record-level license Zenodo displays is set explicitly in `.zenodo.json` (recommend `cc-by-4.0`, since documentation is the bulk of the work), with the tripartite scheme restated in the record description. `.zenodo.json` takes precedence over any license files it finds, which sidesteps the alphabetical lottery entirely.

3.3 Rendered artifacts (enablement insurance)

Mermaid source in markdown is only a diagram *after* a renderer touches it. For the archived record, don't make a future examiner or litigant install tooling: render every doc to PDF (`scripts/render_docs.sh`) into `rendered/`, and include the SVG set. **Every** document, not just the numbered lineage: the executive summary carries diagrams too, and a numbered-only glob shipped its Mermaid blocks as raw code fences from v1.0 to v1.1 before `scripts/check_release.py` was written to catch exactly that. The Zenodo tarball is then fully self-contained — readable with nothing but a PDF viewer. This also protects against markdown-flavor drift over decades.

3.4 The disclosure statement (add to README, verbatim-class)

> **Defensive publication notice.** This work is published as a deliberate public disclosure intended to constitute prior art under 35 U.S.C. § 102 and corresponding provisions worldwide. No patent protection is sought or held by the authors for any subject matter disclosed herein, and the authors intend this dated public record to preclude the patenting of the disclosed subject matter by any party. First published: [date]. Archived with DOI: [concept DOI].

3.5 The parameter register (enablement, machine-readable)

Prose carries an argument; a table carries a *check*. `docs/parameter_register.xlsx` collects every quantitative claim in the lineage into one filterable register — value, unit, confidence grade, gating test, derivation, and the source document section — and adds live calculation sheets that re-derive the headline numbers from named constants (the shared psychrometrics, the doc 00 §4 airflow–moisture model, both sizing chains, the CO_2 ladder, the X12 lift ceiling).

Why this matters for a defensive publication, beyond convenience:

- **Enablement.** A person of ordinary skill can re-solve the design at their own conditions rather than accept ours. Changing the design point on one sheet propagates through every derived figure, which is the difference between a disclosure that can be *practiced* and one that can only be *read*.
- **Verifiability.** Every row names its source section, so any figure can be traced back to the document that asserts it. A register row with no source does not belong in the file.
- **Self-audit.** Re-deriving each published number surfaces the places where a figure and its stated basis have drifted apart. Those observations belong on the register's own `Checks` sheet and, where they touch a published number, in the errata trail — never as a silent correction (§8).

Rules that keep it honest: it is a **derived** artifact and the documents remain authoritative — where the two disagree, the document wins and the discrepancy is recorded. It is **generated, never hand-edited** (`scripts/build_parameter_workbook.py`, MIT), so it is reproducible from source. It carries **no ungraded number**, exactly as doc 00 §9 requires of the prose. And it is regenerated with the PDF set at release time, so a stale register is a release blocker on the same footing as a stale `rendered/`.

3.6 Initialization commands

```
git init slice-cooling && cd slice-cooling
# ...populate per §3.1...
git add -A
git commit -m "v1.0 — lineage reset: docs 00-40, diagrams, rendered set"
git tag -a v1.0 -m "v1.0 defensive publication release"
git remote add origin git@github.com:<user>/slice-cooling.git
git push -u origin main --tags
```

Sign the tag (`git tag -s`) if you have a GPG key on the GitHub account — not load-bearing for prior art, but it strengthens the chain-of-custody narrative. Make the repository **public before** creating the Zenodo record, and never force-push over published history: the git log is part of the evidentiary story even though it isn't the anchor.

4. Zenodo procedure

Two paths exist; use both, in this order.

4.1 First record — manual upload (full control)

The manual path lets you review every metadata field before the DOI is minted, which matters most on the first, anchor record.

1. Create a Zenodo account — log in via **ORCID** (get an ORCID first if you don't have one; it's the persistent author identifier examiners and indexers key on).
2. **New upload.** Attach: the release tarball (`git archive --format=tar.gz v1.0`), the rendered PDF set, and `executive_summary.pdf` as a separate top-level file (Zenodo previews the first PDF — make it the summary).
3. Resource type: **Publication → Technical note** (better indexed for prior-art purposes than "Software"; the record can still contain code).
4. Title, authors (with ORCID), and the **description field = the executive summary abstract** plus the §3.4 disclosure notice plus the keyword block (§5).
5. License: CC-BY-4.0 at record level; restate the tri-license scheme in the description.

6. Publication date: today. Do **not** use embargo or restricted access — either would undercut public accessibility.
7. Optionally reserve the DOI before publishing (button in the draft) so it can be baked into the README of the very tarball you upload — a nice self-referencing touch, but requires re-generating the tarball after reserving.
8. **Publish.** Zenodo mints two DOIs: a **version DOI** (this exact v1.0 record, files frozen forever) and a **concept DOI** (resolves to the latest version). Put the *concept* DOI in the README badge and the *version* DOI in the disclosure notice for v1.0.

After publishing, metadata remains editable but **files do not** — corrections go in as a new version. This immutability is precisely the property that makes the record evidentially strong; the lineage-reset discipline already practiced in this repo maps onto it perfectly.

4.2 Ongoing — GitHub webhook (automation)

For v1.1 onward, enable the integration so every GitHub release archives itself:

1. Zenodo → account settings → **GitHub** → link the GitHub account.
2. Flip the toggle on the repository.
3. Each subsequent GitHub **release** (not a bare tag — releases specifically) triggers an automatic Zenodo deposit under the same concept DOI, with a new version DOI.

Note the integration cannot attach to the existing manual record — the automated releases will form their own concept-DOI chain. Handle this by adding a "related identifiers" entry on both records pointing at each other (*isNewVersionOf / isPreviousVersionOf*), or simply skip the manual record and do v1.0 through the webhook too. Recommendation: **manual for v1.0** (metadata control on the anchor record outweighs the split-chain wrinkle, which the related-identifier links fully repair).

4.3 `.zenodo.json` skeleton

```
{
  "title": "Heat-Driven Desiccant Comfort and Water Platform for Humid Climates (Land and Marine): CaCl2 Liquid-Desiccant and Aluminum-Fumarate MOF Coated-Exchanger Tracks",
  "upload_type": "publication",
  "publication_type": "technicalnote",
  "license": "cc-by-4.0",
  "creators": [{ "name": "<Surname, Given>", "orcid": "0000-0000-0000-0000" }],
  "keywords": [
    "liquid desiccant air conditioning", "LDAC", "calcium chloride brine",
    "dehumidification", "aluminum fumarate", "MIL-53(Al)-FA", "Basolite A520",
    "metal-organic framework", "desiccant-coated heat exchanger", "DCHX",
    "Maisotsenko cycle", "dew-point indirect evaporative cooling",
    "atmospheric water harvesting", "humidification-dehumidification desalination",
    "thermal swing adsorption", "solid amine CO2 scrubbing",
    "solar thermal regeneration", "waste heat recovery", "marine HVAC",
    "enthalpy recovery ventilator", "defensive publication", "prior art"
  ],
  "notes": "Defensive publication. No patents sought or held. Hardware: CERN-OHL-P v2; documentation: CC-BY-4.0; scripts: MIT.",
  "related_identifiers": [
    { "identifier": "https://github.com/<user>/slice-cooling",
      "relation": "isSupplementTo", "resource_type": "software" }
  ]
}
```

5. Metadata for examiner discoverability

The abstract and keywords are the search surface. Discipline:

- Use the **standard literature terminology alongside the canonical repo terms** — an examiner searches "liquid desiccant dehumidification" and "desiccant-coated heat exchanger," not "moisture battery" or "raw-water sink." The executive summary already does this well; keep both vocabularies in the record description.
- Name the specific materials by every alias: aluminum fumarate / AlFu / Basolite A520 / MIL-53(Al)-FA; $CaCl_2$ / calcium chloride brine; Lewatit VP OC 1065-class solid amine.
- Consider listing plausible **CPC classification codes** in the description (e.g., F24F 3/14 and F24F 3/1417 for desiccant air conditioning, F24F 5/0035 for evaporative cooling, B01D 53/26 and 53/28 for sorbent drying/CO_2, C02F 1/04-class for thermal desalination, B63J for shipboard systems). Examiners search by classification; putting the codes in the text makes the record surface in classification-plus-keyword queries. Verify the exact codes against the current CPC scheme before publishing rather than trusting this list.
- Zenodo records are harvested by OpenAIRE, DataCite, and indexed by Google Scholar/Dataset Search — the metadata quality above is what determines whether that indexing does anything.

6. Third-party snapshot procedure (day of publication)

1. **Software Heritage:** https://archive.softwareheritage.org/save/ → "Save Code Now" → paste the GitHub URL. Archives the full git history under a persistent SWHID. Repeat after major releases (it also crawls GitHub periodically on its own).
2. **Internet Archive:** https://web.archive.org/save/ → snapshot (a) the repo root, (b) the release page, (c) the Zenodo record page. Three URLs, two minutes.
3. **OpenTimestamps (optional):** `ots stamp v1.0.tar.gz` → commit the `.ots` proof file to the repo in the next release. Free, and converts "trust Zenodo's clock" into "trust the Bitcoin blockchain's clock" for anyone who demands cryptographic proof.

7. Optional examiner-channel deposit

TDCommons (tdcommons.org, operated with Google's backing, free): accepts defensive publications directly, feeds databases used in examiner search.

The deposit is a single PDF — the executive summary with the DOI reference is the right artifact; it points searchers at the full enabling record. One-time, ~30 minutes. Recommended.

Research Disclosure (researchdisclosure.com, paid): the only venue in the WIPO PCT minimum documentation list, meaning international examiners are *required* to have searched it. Hold in reserve; deploy if commercial patenting activity appears in this space.

8. Version discipline going forward

- Every substantive lineage change = git tag + GitHub release + (webhook) Zenodo version. Errata continue to accumulate visibly in the documents, per repo doctrine — the version chain on Zenodo *is* the public correction trail.
- Never delete or force-rewrite published history; supersede, as the lineage already does.
- The disclosure date that matters for any given finding is the date of the **first version containing it** — a reason to release reasonably often when new findings (X11+, F6+) land, rather than batching for years.
- Bench results, when they arrive, upgrade claims from sizing-grade to measured — publish those versions promptly; measured data is the strongest possible enablement.
- The parameter register (§3.5) is regenerated with the PDF set on every release, and any figure it cannot reproduce from its stated basis is resolved before the tag — either as a clarifying edit, or, where a published number moves, as an erratum. The register's `Checks` sheet is the working list; the errata trail is where the resolution lands.

9. Publication-day checklist

Run `python3 scripts/check_release.py` first — it mechanically enforces the structural half of this list (document version discipline and footers, PDF and register freshness, no raw Mermaid in any PDF, version and DOI agreement across README/ `CITATION.cff` / `.zenodo.json`, the §3.2 license layout, and the verbatim rule on the wide-diagram overrides). It exits non-zero on any failure. The items below that need human judgement stay manual.

- ☐ `python3 scripts/check_release.py` passes
- ☐ ORCID obtained; GitHub repo public with §3.1 structure
- ☐ `rendered/` PDF set generated and spot-checked (Unicode, diagrams)
- ☐ `docs/parameter_register.xlsx` regenerated (§3.5); formulas recalculate clean and every open item on its `Checks` sheet is resolved or recorded
- ☐ `LICENSE.md` scope map + three texts in `LICENSES/`
- ☐ `.zenodo.json` + `CITATION.cff` committed
- ☐ Disclosure statement in README (date + DOI placeholders)
- ☐ `git tag -a v1.0` pushed
- ☐ Zenodo manual record published; concept + version DOIs recorded
- ☐ README updated with DOI badge → commit → (optionally) v1.0.1 tag so the in-repo README carries its own DOI
- ☐ Software Heritage save + 3× Wayback snapshots
- ☐ GitHub↔Zenodo webhook enabled for future releases
- ☐ TDCommons deposit of the executive summary (recommended)
- ☐ `ots stamp` on the release tarball (optional)

Part of an open defensive-publication release: hardware CERN-OHL-P v2, text CC-BY-4.0, scripts MIT. No patents sought or held.

Version history

- **v1.0** — Initial disclosure-strategy document: venue stack, repo formation, Zenodo manual + webhook procedures, metadata discipline, snapshot and examiner-channel procedures, ongoing version discipline.
- **v1.2** — §9 now opens with `scripts/check_release.py`, which turns the structural half of the checklist into a gate that exits non-zero; §3.3 records that the renderer covers every document, after the executive summary was found to have shipped un-rendered Mermaid source from v1.0 to v1.1.
- **v1.1** — §3.5 added: the generated parameter register (`docs/parameter_register.xlsx`) recorded as machine-readable enablement, with its derived-artifact and no-ungraded-number rules; §3.1 structure block and the §9 checklist updated to carry it; §8 gains the release-time reconciliation rule (a figure the register cannot reproduce is resolved before the tag — clarifying edit or erratum, never silently); former §3.5 renumbered to §3.6.